

MANUAL COMPLETO DE PROGRAMACIÓN II

Grado en Ingeniería Informática - 1º Curso

Este documento unifica todos los temas del plan de estudio de orientación a objetos en Java, especificación y corrección formal de código, recursividad, análisis de complejidad temporal y TADs lineales en un único manual para facilitar su lectura, impresión o conversión a formatos como PDF.

Índice General del Manual

- **Bloque 1: Programación Orientada a Objetos Robusta (Semanas 1 a 6)**
 - [Tema 1: Transición a Java y Gestión de Memoria \(Stack vs. Heap\)](#)
 - [Tema 2: Clases y Encapsulamiento](#)
 - [Tema 3: Mecanismos de Reutilización: Herencia](#)
 - [Tema 4: Polimorfismo, Enlace Dinámico e Interfaces](#)
 - [Tema 5: Gestión de Excepciones](#)
 - [Tema 6: Genericidad](#)
- **Bloque 2: Rigor Formal y Recursividad (Semanas 7 a 11)**
 - [Tema 7: Especificación Formal de Código y Diseño por Contrato](#)
 - [Tema 8: Algoritmos Recursivos](#)
 - [Tema 9: Demostración de Corrección en Bucles \(Invariantes y Cotas\)](#)
 - [Tema 10: Demostración de Corrección en Algoritmos Recursivos](#)
- **Bloque 3: Eficiencia y Estructuras de Datos (Semanas 12 a 15)**
 - [Tema 11: Complejidad y Coste Algorítmico](#)
 - [Tema 12: Análisis de Coste en Estructuras Iterativas](#)
 - [Tema 13: Análisis de Coste en Algoritmos Recursivos](#)
 - [Tema 14: Tipos Abstractos de Datos \(TADs\) Lineales](#)
- **Secciones Finales**
 - [Glosario de Términos](#)
 - [Bibliografía Recomendada](#)

Tema 1: Transición a Java y Gestión de Memoria (Stack vs. Heap)

Al programar en lenguajes como C, el desarrollador tiene el control total (y la responsabilidad) de la gestión de memoria utilizando punteros y funciones como `malloc()` y `free()`. La transición a Java introduce una capa de abstracción crítica: la **Máquina Virtual de Java (JVM)**. En Java no existen punteros explícitos ni aritmética de punteros, y la memoria se gestiona de forma automática. Para evitar errores y comprender la ejecución del software, es imprescindible entender cómo organiza la JVM la memoria en dos áreas fundamentales: el **Stack (Pila)** y el **Heap (Montículo)**.

1. Stack vs. Heap: Organización Espacial de la Memoria

La JVM divide la memoria en dos regiones con propósitos de diseño muy diferentes:

A. El Stack (Pila de llamadas)

Es una memoria de acceso rápido estructurada bajo el principio LIFO (Last-In, First-Out).

- **Contenido:** Almacena variables locales de tipo primitivo (como `int`, `double`, `boolean`) y las **referencias** (direcciones de memoria) a los objetos que residen en el Heap. También guarda los marcos de activación de los métodos en ejecución (parámetros y direcciones de retorno).
- **Ciclo de vida:** Su gestión es automática y ligada al flujo de control. Al invocar un método, se crea su marco de llamada en el tope del Stack; al terminar el método, el marco se destruye de inmediato y la memoria de sus variables locales se libera.

B. El Heap (Montículo de objetos)

Es una gran área de memoria de acceso dinámico y global.

- **Contenido:** Almacena **todos los objetos** creados en el programa (mediante la palabra clave `new`), incluyendo sus atributos (incluso si estos atributos son primitivos) y todos los arrays.

- **Ciclo de vida:** No está acotado por el ámbito de un método. Un objeto permanece en el Heap mientras exista al menos una referencia activa en el Stack que apunte a él.

2. Referencias y su representación gráfica

En Java, cuando declaramos una variable de tipo objeto, por ejemplo:

```
Persona p = new Persona("Ana");
```

No estamos guardando el objeto **Persona** dentro de la variable **p**. En su lugar, la variable **p** reside en el Stack y almacena una **referencia** (un puntero implícito seguro) que apunta a la ubicación física del objeto real **Persona** creado en el Heap.

Representación Espacial en Memoria (ASCII)

Si ejecutamos el siguiente bloque de código:

```
Persona p1 = new Persona("Ana");
Persona p2 = p1;
Persona p3 = new Persona("Luis");
```

La distribución física de la memoria de la JVM se estructurará de la siguiente forma:

STACK (Pila)			HEAP (Montículo)	
p1	ref @001	----->	Persona	
			- nombre: "Ana"	
p2	ref @001	-----/		
p3	ref @002	----->	Persona	
			- nombre: "Luis"	

Consecuencias Críticas de la Copia de Referencias:

- La asignación **p2 = p1** **no copia el objeto** en el Heap; únicamente copia la dirección de memoria en el Stack.
- Tanto **p1** como **p2** apuntan al mismo y único objeto **"Ana"**.

- Si modificamos el nombre del objeto a través de `p2.setNombre("Clara")`, la llamada a `p1.getNombre()` devolverá "Clara", ya que el objeto subyacente es compartido.
-

3. Ciclo de Vida de un Objeto y Garbage Collector (GC)

Ciclo de Vida

1. **Creación:** Se reserva espacio en el Heap mediante `new`, se ejecuta el constructor y se devuelve la referencia.
2. **Uso:** Se accede a sus métodos y atributos mediante su referencia en el Stack.
3. **Inalcanzabilidad:** Un objeto se vuelve "inalcanzable" (huérfano) cuando no existe ninguna referencia activa en el Stack (ni en otros objetos del Heap) que apunte a él.

El Recolector de Basura (Garbage Collector)

El Garbage Collector (GC) es un hilo de ejecución de baja prioridad de la JVM que se ejecuta en segundo plano de manera automática.

- **Función:** Identifica y destruye los objetos inalcanzables en el Heap para liberar su espacio de memoria.
 - **Funcionamiento:** Aunque existen varios algoritmos (como *Mark & Sweep*), la premisa es que el GC recorre las raíces (variables del Stack) y marca todo lo alcanzable. Lo que quede sin marcar se barre (se libera).
 - **Invocación:** El desarrollador **no puede forzar** la recolección de basura. Llamar a `System.gc()` es simplemente una sugerencia a la JVM, la cual puede ser ignorada.
-

4. Ejemplo Práctico en Java y su Traza de Memoria

Analicemos el ciclo de vida y la pérdida de referencias con este programa:

```
public class PruebaMemoria {  
    public static void main(String[] args) {  
        // Paso 1: Creación de dos objetos  
        Persona a = new Persona("Carlos");  
        Persona b = new Persona("Marta");  
    }  
}
```

```
// Paso 2: Redirección de referencia
a = b;

// Paso 3: Pérdida total de referencias
b = null;
}
}
```

Traza Paso a Paso de la Memoria:

1. Tras Paso 1:

- **Stack:** Variable **a** guarda ref @010. Variable **b** guarda ref @020.
- **Heap:** Objeto "Carlos" en @010. Objeto "Marta" en @020.

2. Tras Paso 2 (**a = b**):

- **Stack:** Variable **a** ahora guarda ref @020. Variable **b** guarda ref @020.
- **Heap:** El objeto "Carlos" en @010 queda **sin referencias**. Se vuelve inalcanzable. Es candidato para ser destruido por el Garbage Collector en su próxima pasada.

3. Tras Paso 3 (**b = null**):

- **Stack:** Variable **a** mantiene ref @020. Variable **b** pasa a ser **null** (no apunta a nada).
- **Heap:** El objeto "Marta" en @020 **sigue vivo** porque la referencia **a** todavía apunta a él.

5. Ejercicios Resueltos

Ejercicio 1

Dado el siguiente código en Java:

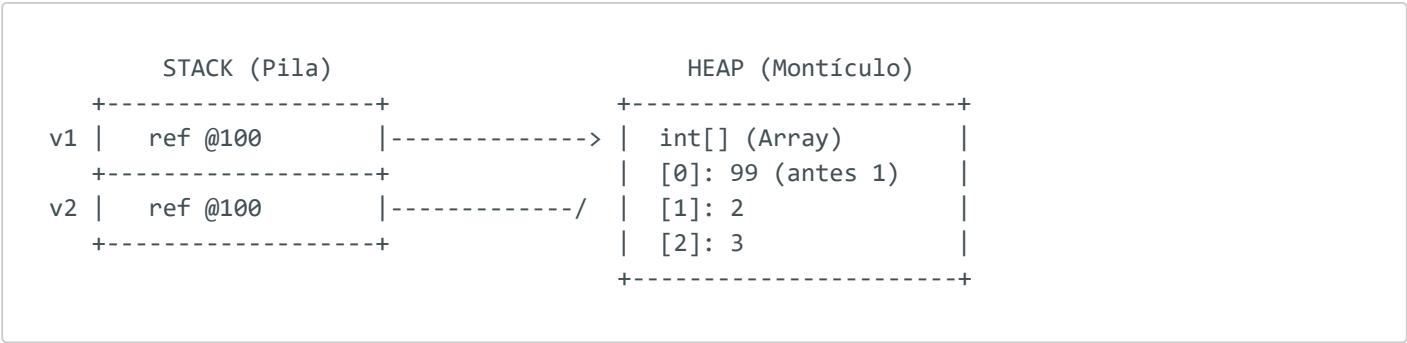
```
public class Ejercicio {
    public static void main(String[] args) {
        int[] v1 = {1, 2, 3};
        int[] v2 = v1;
        v2[0] = 99;
        System.out.println(v1[0]);
    }
}
```

1. Explicar razonadamente qué se imprimirá por consola.
2. Dibujar el mapa de memoria (Stack vs. Heap) durante la línea **v2[0] = 99;**.

Solución:

1. **Explicación:** Se imprimirá 99. En Java, los arrays son tratados como objetos y residen en el Heap. La variable `v1` en el Stack guarda la referencia al array `{1, 2, 3}` en el Heap. La línea `int[] v2 = v1;` copia dicha referencia en `v2`. Por tanto, `v1` y `v2` apuntan exactamente al mismo array en el Heap. Al modificar el primer elemento mediante `v2[0] = 99;`, estamos alterando el array compartido, cambio que es inmediatamente visible a través de `v1`.

2. Mapa de Memoria:



Tema 2: Clases y Encapsulamiento

La programación orientada a objetos (POO) busca modelar el mundo real o conceptos lógicos mediante entidades autónomas llamadas **objetos**. En este tema estudiaremos cómo se definen estas entidades mediante las **clases**, cómo interactúan sus componentes y cómo el principio de **encapsulamiento** protege la integridad del estado interno de los objetos.

1. Clases, Objetos y Miembros

- **Clase:** Es el plano, molde o plantilla que define las propiedades (atributos) y el comportamiento (métodos) que tendrán todos los objetos de ese tipo. Reside en la zona de metadatos de la JVM (Method Area).
- **Objeto:** Es una instancia física concreta de una clase, creada en tiempo de ejecución en el Heap.

Miembros de Instancia vs. Miembros Estáticos (**static**)

1. Miembros de Instancia (por defecto):

- Cada objeto en el Heap tiene su propia copia de los atributos.
- Para acceder a ellos se requiere una instancia concreta (**objeto.atributo**, **objeto.metodo()**).

2. Miembros Estáticos (**static**):

- Pertenecen a la clase en sí, no a ningún objeto particular.
 - Solo existe **una única copia** en memoria para toda la aplicación, compartida por todas las instancias de esa clase.
 - Se pueden invocar directamente mediante el nombre de la clase (**Clase.atributo**, **Clase.metodo()**).
 - **Limitación:** Un método estático no tiene referencia implícita **this** y **no puede acceder directamente** a atributos o métodos de instancia, ya que estos últimos necesitan un objeto concreto para existir.
-

2. Constructores y Sobrecarga

Un **constructor** es un método especial que se invoca automáticamente al crear un objeto con la palabra clave **new**. Su función es reservar memoria e inicializar el estado del objeto.

- Tienen el mismo nombre exacto que la clase y no especifican ningún tipo de retorno (ni siquiera **void**).
- **Constructor por defecto**: Si no definimos ningún constructor, el compilador genera uno vacío sin parámetros de forma automática. Si definimos al menos un constructor propio, el automático se pierde.
- **Sobrecarga de Constructores**: Podemos definir múltiples constructores con diferentes firmas (lista de parámetros).
- **Uso de **this(...)****: Permite que un constructor invoque a otro constructor de la misma clase para evitar duplicidad de código. Debe ser la primera instrucción de la función.

3. Modificadores de Acceso y Ocultación de Datos

Los modificadores de acceso determinan la visibilidad de los atributos y métodos de una clase desde otras partes del código:

Modificador	Dentro de la Clase	Mismo Paquete	Subclases (Herencia)	En cualquier parte
private	Sí	No	No	No
<i>Sin modificador</i>	Sí	Sí	No	No
protected	Sí	Sí	Sí	No
public	Sí	Sí	Sí	Sí

4. El Principio de Encapsulamiento

El encapsulamiento consiste en agrupar los atributos (estado) y métodos (comportamiento) que operan sobre ellos dentro de una clase, y **ocultar los detalles de implementación** al exterior.

Regla de Oro del Encapsulamiento:

1. **Todos los atributos deben ser privados (`private`)**. Esto evita que el código externo modifique directamente los datos de un objeto de forma descontrolada.
2. **El acceso se gestiona mediante métodos públicos `get` (lectura) y `set` (escritura)**.
3. **Los setters deben validar los datos**: Esto garantiza la consistencia del objeto.

Ejemplo de Violación y Corrección de Consistencia:

- *Violado*: Si el atributo `edad` es público, un programador externo puede hacer `persona.edad = -5;`, dejando al objeto en un estado biológicamente imposible.
- *Corregido*:

```
public class Persona {
    private int edad;

    public void setEdad(int edad) {
        if (edad >= 0 && edad <= 120) {
            this.edad = edad; // Asignación segura
        } else {
            throw new IllegalArgumentException("Edad no válida");
        }
    }
}
```

5. Visualización en Memoria de Miembros Estáticos

Supongamos que tenemos la clase `CuentaBancaria` con un atributo estático `tasaInteres`:

```
CuentaBancaria c1 = new CuentaBancaria(1000);
CuentaBancaria c2 = new CuentaBancaria(2000);
```

Mapa de Memoria en la JVM (ASCII):

	STACK (Pila)		HEAP (Montículo)
c1	ref @001	----->	CuentaBancaria
			- saldo: 1000
c2	ref @002	----->	CuentaBancaria

```
| - saldo: 2000 |
+-----+

METHOD AREA (Metadatos de Clase)
+-----+
| Clase CuentaBancaria |
| [static] tasaInteres: 2.5% |
+-----+
```

- **saldo** se replica en cada objeto del Heap.
- **tasaInteres** reside en el Method Area (única en memoria), accesible y compartida por todas las referencias.

6. Ejercicios Resueltos

Ejercicio 1

Diseñar una clase **SensorTemperatura** que represente un sensor físico. Debe cumplir:

1. El atributo **temperatura** (en Celsius) debe estar encapsulado. No puede ser inferior a -273.15°C (cero absoluto).
2. Debe tener un constructor que reciba la temperatura inicial y otro por defecto que la inicialice a 0°C utilizando delegación de constructores **this(...)**.
3. Debe existir un atributo estático **numSensores** que lleve la cuenta de cuántas instancias se han creado.

Solución:

```
public class SensorTemperatura {
    // Atributos de instancia (privados para encapsular)
    private double temperatura;

    // Atributo estático (compartido para conteo global)
    private static int numSensores = 0;

    // Constructor con parámetros
    public SensorTemperatura(double temperaturaInicial) {
        setTemperatura(temperaturaInicial); // Valida y asigna
        numSensores++; // Incrementa contador global de instancias
    }

    // Constructor por defecto (delega en el principal)
    public SensorTemperatura() {
        this(0.0); // Invoca al constructor superior con el valor 0.0
    }

    // Getter de temperatura
    public double getTemperatura() {
```

```
        return this.temperatura;
    }

    // Setter con validación estricta de consistencia física
    public void setTemperatura(double temperatura) {
        if (temperatura >= -273.15) {
            this.temperatura = temperatura;
        } else {
            throw new IllegalArgumentException("La temperatura no puede ser menor al cero absoluto.");
        }
    }

    // Getter del contador estático (método estático porque no depende de una instancia)
    public static int getNumSensores() {
        return numSensores;
    }
}
```

(Nota: El getter de *numSensores* se declara estático, lo que permite consultar el total de sensores creados mediante *SensorTemperatura.getNumSensores()* sin necesidad de tener un objeto creado).

Tema 3: Mecanismos de Reutilización: Herencia

La herencia es uno de los pilares de la programación orientada a objetos. Permite crear una nueva clase (subclase o clase derivada) a partir de una clase existente (superclase o clase base), heredando sus atributos y métodos. Esto evita la duplicación de código y permite establecer una relación jerárquica de tipo "es-un" (por ejemplo, un **Estudiante es una Persona**).

1. Conceptos Fundamentales de la Herencia

En Java, la herencia se implementa mediante la palabra clave **extends**:

```
public class Estudiante extends Persona {  
    // Estudiante hereda todos los miembros no privados de Persona  
}
```

Reglas Clave en Java:

1. **Herencia Simple:** En Java **no existe la herencia múltiple de clases**. Una clase solo puede extender directamente a una única superclase (es decir, una sola rama paterna).
 2. **La clase Object:** Si no especificamos ninguna superclase con **extends**, la clase hereda de forma automática e implícita de **java.lang.Object** (la raíz de todas las jerarquías en Java).
 3. **Constructores:** Los constructores **no se heredan**. Cada subclase debe definir sus propios constructores.
-

2. El Modificador de Acceso **protected**

El modificador **protected** proporciona un nivel intermedio de encapsulación:

- Un miembro **protected** (atributo o método) es accesible por la propia clase, por las clases que estén en el **mismo paquete** y por las **subclases** (incluso si estas se encuentran en paquetes diferentes).

- *Buenas prácticas*: Evita hacer atributos `protected` directamente si deseas un encapsulamiento fuerte. Es preferible mantener los atributos `private` en la clase base y exponer métodos de acceso `protected` si es necesario que las subclases los manipulen.
-

3. Sobreescritura de Métodos (Method Overriding)

La **sobreescritura** ocurre cuando una subclase define un método con la **misma firma** (nombre, tipo de retorno y parámetros) que un método de su superclase, proporcionando una implementación específica y adaptada a su comportamiento.

La Anotación `@Override`

Es una directiva para el compilador. No cambia el comportamiento del programa, pero es fundamental por dos razones:

1. **Seguridad**: Si cometemos un error de tipografía al sobreescribir (por ejemplo, escribir `toString()` en lugar de `toString()`), el compilador generará un error indicando que el método no existe en la superclase.
2. **Legibilidad**: Documenta claramente que el método está redefiniendo un comportamiento heredado.

Diferencia clave con la **sobrecarga**:

- *Sobreescritura*: Mismo nombre, mismos parámetros, distinta clase (relación de herencia).
 - *Sobrecarga*: Mismo nombre, distintos parámetros, misma clase.
-

4. La Referencia `super`

La palabra clave `super` hace referencia directa a la superclase inmediata de la clase actual. Se utiliza en dos escenarios:

A. Invocación de Constructores del Padre

En el constructor de una subclase, podemos llamar al constructor de la superclase usando `super(...)`:

```
public class Estudiante extends Persona {  
    private String matricula;  
  
    public Estudiante(String nombre, int edad, String matricula) {  
        super(nombre, edad); // Llama al constructor de Persona. DEBE SER LA PRIMERA LÍNEA.  
        this.matricula = matricula;  
    }  
}
```

Si no escribimos explícitamente `super(...)` en la primera línea de un constructor, el compilador inserta de forma invisible la llamada al constructor sin parámetros del padre: `super();`. Si el padre no tiene constructor sin parámetros, se producirá un error de compilación.

B. Invocación de Métodos del Padre

Permite invocar la versión del método que existe en la superclase, lo que resulta útil cuando queremos complementar la funcionalidad heredada en lugar de reemplazarla por completo:

```
@Override  
public void mostrarInformacion() {  
    super.mostrarInformacion(); // Ejecuta la impresión básica del padre (nombre, edad)  
    System.out.println("Matrícula: " + this.matricula); // Añade el campo específico  
}
```

5. Orden de Construcción en Memoria

Cuando creamos una instancia de una subclase (`new Estudiante()`), la JVM ejecuta los constructores en cadena de arriba hacia abajo de la jerarquía:

1. Se reserva memoria en el Heap para el objeto completo (que contendrá tanto los atributos de `Persona` como los de `Estudiante`).
2. Se ejecuta el constructor de `Object`.
3. Se ejecuta el constructor de `Persona` (inicializa atributos heredados).
4. Se ejecuta el constructor de `Estudiante` (inicializa atributos específicos).

6. Ejercicios Resueltos

Ejercicio 1

Diseñar la estructura de clases para un sistema de personal universitario:

1. Crear la clase base `Empleado` con los atributos `nombre` (String) y `sueldoBase` (double). Debe tener un método `calcularSueldo()` que simplemente retorne `sueldoBase`.
2. Crear la subclase `Profesor` que hereda de `Empleado` y añade el atributo `horasExtras` (int) y la constante estática `PRECIO_HORA = 20.0`. Sobreescibir `calcularSueldo()` para que devuelva el sueldo base más el pago por las horas extras.
3. Implementar los constructores correspondientes utilizando `super(...)` y la anotación `@Override`.

Solución:

```
// Clase Base
public class Empleado {
    private String nombre;
    private double sueldoBase;

    public Empleado(String nombre, double sueldoBase) {
        this.nombre = nombre;
        setSueldoBase(sueldoBase);
    }

    public String getNombre() {
        return nombre;
    }

    public double getSueldoBase() {
        return sueldoBase;
    }

    public void setSueldoBase(double sueldoBase) {
        if (sueldoBase >= 0) {
            this.sueldoBase = sueldoBase;
        } else {
            throw new IllegalArgumentException("El sueldo no puede ser negativo");
        }
    }

    public double calcularSueldo() {
        return this.sueldoBase;
    }
}

// Clase Derivada
public class Profesor extends Empleado {
    private int horasExtras;
    public static final double PRECIO_HORA = 20.0;

    public Profesor(String nombre, double sueldoBase, int horasExtras) {
        super(nombre, sueldoBase); // Delega la inicialización del nombre y sueldoBase al
padre
        setHorasExtras(horasExtras);
    }
}
```

```
public int getHorasExtras() {  
    return horasExtras;  
}  
  
public void setHorasExtras(int horasExtras) {  
    if (horasExtras >= 0) {  
        this.horasExtras = horasExtras;  
    } else {  
        throw new IllegalArgumentException("Las horas extras no pueden ser negativas");  
    }  
}  
  
// Sobreescritura del método calcularSueldo  
@Override  
public double calcularSueldo() {  
    // Aprovechamos el sueldo base calculado por el padre y le sumamos el extra de  
profesor  
    return super.calcularSueldo() + (this.horasExtras * PRECIO_HORA);  
}  
}
```

Tema 4: Polimorfismo, Enlace Dinámico e Interfaces

El polimorfismo y las interfaces permiten diseñar sistemas de software altamente extensibles y desacoplados. Mediante estas herramientas, podemos escribir código que opere sobre abstracciones generales (como "FiguraGeometrica" o "ConexionBD") sin preocuparnos por las implementaciones concretas que existirán en el futuro, las cuales se resolverán dinámicamente en tiempo de ejecución.

1. Polimorfismo y Tipado (Estático vs. Dinámico)

El **polimorfismo** es la propiedad por la cual una variable de referencia puede apuntar a objetos de diferentes tipos (clases) dentro de una jerarquía de herencia.

Para comprender el polimorfismo, debemos distinguir entre dos conceptos de tipo:

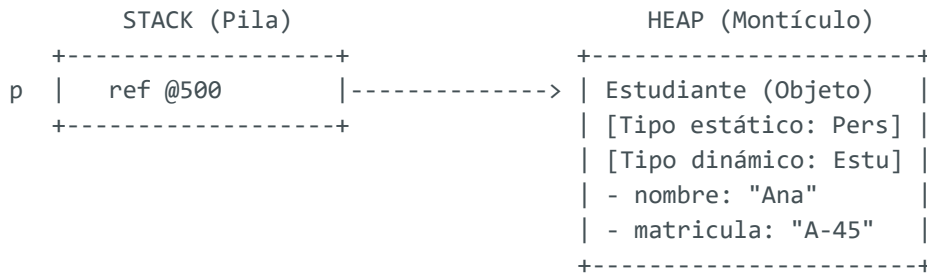
1. **Tipo Estático (o de Declaración)**: Es el tipo con el que se declara la variable de referencia en el código. Se evalúa en tiempo de compilación y determina qué métodos **está permitido invocar** sobre esa variable.
 2. **Tipo Dinámico (o de Ejecución)**: Es el tipo real del objeto creado en el Heap al que apunta la referencia. Se evalúa en tiempo de ejecución y determina qué implementación del método **se ejecutará realmente**.
-

2. Enlace Dinámico (Dynamic Binding)

El **enlace dinámico** es el mecanismo por el cual la JVM decide en tiempo de ejecución qué versión de un método sobrescrito debe invocar. La regla de oro es: **la JVM busca el método en base al tipo dinámico del objeto**.

Visualización en Memoria de **Persona p = new Estudiante();** (ASCII):

Supongamos que la clase **Estudiante** sobrescribe el método **mostrarInformacion()** de **Persona**.



- **¿Es válida la declaración?** Sí, porque un **Estudiante** es *una* **Persona** (Upcasting automático).
- **¿Qué pasa si llamamos a `p.mostrarInformacion()`?** El compilador comprueba que `mostrarInformacion()` existe en el tipo estático **Persona** (pasa la compilación). En tiempo de ejecución, la JVM sigue la referencia `@500` al Heap, detecta que el objeto real es un **Estudiante** y ejecuta la versión sobrescrita de **Estudiante**.
- **¿Qué pasa si llamamos a `p.getMatricula()`?** Da **error de compilación**. Aunque el objeto en el Heap tiene matrícula, el tipo estático de la referencia es **Persona**, y la clase **Persona** no posee el método `getMatricula()`.

3. Conversión de Tipos (Casting) y el operador **instanceof**

Para acceder a métodos específicos del tipo dinámico, debemos cambiar el tipo estático de la referencia mediante un **Casting**:

- **Upcasting (Ascendente)**: Convertir la referencia a una superclase. Es automático e intrínsecamente seguro:

```
Persona p = new Estudiante();
```

- **Downcasting (Descendente)**: Convertir la referencia a una subclase. Debe ser explícito y puede ser peligroso:

```
Estudiante e = (Estudiante) p; // Ahora e puede llamar a getMatricula()
```

Si el objeto apuntado por **p** no fuera realmente un **Estudiante** en tiempo de ejecución, el programa lanzaría un error crítico: **ClassCastException**.

El operador `instanceof`

Para realizar un downcasting seguro, verificamos previamente la compatibilidad del tipo dinámico:

```
if (p instanceof Estudiante) {  
    Estudiante e = (Estudiante) p;  
    System.out.println("Matrícula: " + e.getMatricula());  
}
```

4. Clases Abstractas

Una clase declarada con la palabra clave `abstract` representa un concepto incompleto.

- **Instanciación:** No se pueden crear objetos de una clase abstracta (`new ClaseAbstracta()` produce un error de compilación).
- **Métodos Abstractos:** Pueden declararse sin cuerpo (`public abstract void dibujar();`). Las subclasses no abstractas (concretas) están obligadas a sobrescribir e implementar todos los métodos abstractos heredados.
- Pueden contener atributos de instancia y métodos normales ya implementados.

5. Interfaces

Una **interfaz** (`interface`) representa un contrato de diseño puro. Define un conjunto de operaciones que una clase "se compromete" a realizar.

Características en Java:

1. **Sin Atributos de Instancia:** Solo pueden definir constantes públicas, implícitamente `public static final`.
2. **Métodos:** Todos los métodos son implícitamente públicos y abstractos (`public abstract`), excepto los métodos con implementación por defecto (`default`) o estáticos (`static`) introducidos en Java 8+.
3. **Herencia Múltiple:** Una clase en Java solo puede extender de una clase base, pero puede implementar múltiples interfaces (`implements InterfazA, InterfazB`). Esto permite heredar comportamientos sin los problemas de ambigüedad de la herencia múltiple de clases.

6. Ejercicios Resueltos

Ejercicio 1

Diseñar un sistema de pagos utilizando interfaces y polimorfismo:

1. Definir la interfaz `MetodoPago` con un método `procesarPago(double importe)`.
2. Implementar la clase `TarjetaCredito` que implementa `MetodoPago` y almacena el número de tarjeta y la comisión (1.5%).
3. Implementar la clase `Paypal` que implementa `MetodoPago` y almacena el correo electrónico.
4. Crear una clase `ProcesadorPagos` con un método estático `realizarTransaccion(MetodoPago metodo, double importe)` que ilustre el polimorfismo invocando el cobro.

Solución:

```
// Interfaz (Contrato)
public interface MetodoPago {
    void procesarPago(double importe);
}

// Implementación 1
public class TarjetaCredito implements MetodoPago {
    private String numeroTarjeta;
    private static final double COMISION = 0.015; // 1.5%

    public TarjetaCredito(String numeroTarjeta) {
        this.numeroTarjeta = numeroTarjeta;
    }

    @Override
    public void procesarPago(double importe) {
        double total = importe * (1 + COMISION);
        System.out.println("Pago de " + total + " EUR procesado con Tarjeta " +
            numeroTarjeta + " (comisión de " + (importe * COMISION) + " EUR incluida).");
    }
}

// Implementación 2
public class Paypal implements MetodoPago {
    private String email;

    public Paypal(String email) {
        this.email = email;
    }

    @Override
    public void procesarPago(double importe) {
        System.out.println("Pago de " + importe + " EUR procesado a través de PayPal para el
```

```

    usuario " + email + ".");
    }
}

// Procesador que aprovecha el Polimorfismo
public class ProcesadorPagos {
    // El primer parámetro es polimórfico: acepta CUALQUIER objeto que implemente MetodoPago
    public static void realizarTransaccion(MetodoPago metodo, double importe) {
        System.out.println("Iniciando transacción...");

        // Enlace Dinámico: la JVM decidirá si invoca la versión de TarjetaCredito o de
        Paypal
        // en base al tipo dinámico del objeto que reciba en el parámetro 'metodo'
        metodo.procesarPago(importe);

        System.out.println("Transacción finalizada.\n");
    }

    public static void main(String[] args) {
        MetodoPago tarjeta = new TarjetaCredito("1234-5678-9012-3456");
        MetodoPago cuentaPaypal = new Paypal("usuario@correo.com");

        // Realizamos cobros de 100 EUR con diferentes métodos
        realizarTransaccion(tarjeta, 100.0);
        realizarTransaccion(cuentaPaypal, 100.0);
    }
}

```

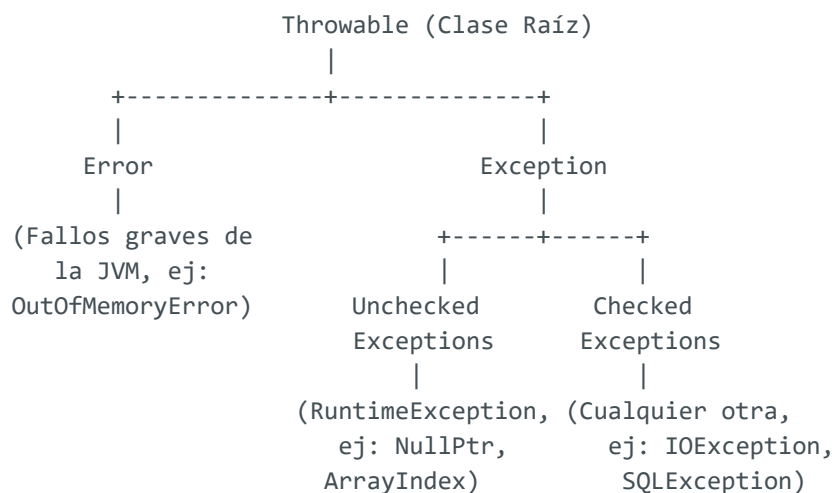
(Nota: El método *realizarTransaccion* está totalmente desacoplado de las implementaciones concretas de tarjetas o cuentas de PayPal. Si en el futuro añadimos una clase *Criptomoneda* que implemente *MetodoPago*, funcionará de inmediato con el procesador de pagos existente sin cambiar una sola línea de código).

Tema 5: Gestión de Excepciones

Un programa de software de calidad industrial debe ser tolerante a fallos. Si un fichero no existe, la red se cae, o un usuario introduce datos inválidos, el programa no debe cerrarse de forma abrupta ("romperse"). La **gestión de excepciones** es el mecanismo estructurado que proporciona Java para interceptar eventos anómalos en tiempo de ejecución, procesarlos de forma segura y garantizar el cierre correcto de los recursos del sistema.

1. Jerarquía de Excepciones en Java

En Java, todas las excepciones son objetos. La raíz de toda la estructura es la clase **Throwable**, que se divide en dos grandes ramas:



1. **Error**: Representa fallos catastróficos de hardware o de la propia JVM (como **OutOfMemoryError** o **StackOverflowError**). La aplicación **no debe intentar capturarlos**, ya que la recuperación es inviable.
2. **Exception**: Errores generados por el programa que sí pueden capturarse y gestionarse. Se dividen a su vez en excepciones **verificadas** y **no verificadas**.

2. Excepciones Verificadas (Checked) vs. No Verificadas (Unchecked)

A. Excepciones Verificadas (Checked Exceptions)

Son subclases directas de `Exception` (excluyendo a `RuntimeException`).

- **Filosofía:** Representan fallos previsibles y ajenos al control directo del programador (fallos de red, archivos no encontrados, etc.).
- **Control de compilación:** El compilador obliga a gestionarlas mediante la regla **"Handle or Declare"**: el desarrollador debe capturarlas en un bloque `try-catch` o declarar su propagación en la firma del método mediante la cláusula `throws`.
- *Ejemplos:* `IOException`, `FileNotFoundException`, `SQLException`.

B. Excepciones No Verificadas (Unchecked o Runtime Exceptions)

Son subclases de `RuntimeException`.

- **Filosofía:** Representan errores de lógica del programador o fallos en el diseño del código.
- **Control de compilación:** El compilador no obliga a capturarlas ni a declararlas en la firma. Se asume que el desarrollador debe prevenirlas programando correctamente.
- *Ejemplos:* `NullPointerException` (intentar usar una referencia `null`), `ArrayIndexOutOfBoundsException` (índice de array fuera de rango), `ArithmeticException` (división por cero).

3. Captura y Liberación de Recursos: `finally` vs. `try-with-resources`

Bloque `try-catch-finally`

- `try`: Contiene el código susceptible de lanzar excepciones.
- `catch`: Captura un tipo específico de excepción y define la respuesta. Se pueden encadenar múltiples bloques `catch` (del más específico al más genérico).
- `finally`: Bloque opcional. **Se ejecuta siempre**, ocurra o no una excepción. Se utiliza para liberar recursos (cerrar ficheros o conexiones a bases de datos).

```
// Estructura clásica
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("datos.txt"));
    System.out.println(br.readLine());
} catch (IOException e) {
    System.err.println("Error de lectura: " + e.getMessage());
} finally {
    // Liberar recurso manualmente
```

```
if (br != null) {  
    try { br.close(); } catch (IOException ex) { /* ignorar */ }  
}
```

El bloque **try-with-resources** (Recomendado)

Introducido en Java 7, simplifica la liberación de recursos. Cualquier recurso que implemente la interfaz `java.lang.AutoCloseable` se declara dentro del paréntesis del `try`. La JVM garantiza el **cierre automático** del recurso al salir del bloque, ocurra o no un error, eliminando por completo la necesidad del bloque `finally` repetitivo:

```
// Estructura moderna y limpia con try-with-resources  
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {  
    System.out.println(br.readLine());  
} catch (IOException e) {  
    System.err.println("Error de lectura: " + e.getMessage());  
}
```

4. Lanzamiento (**throw**) y Propagación (**throws**)

- **throw**: Se utiliza para lanzar explícitamente una instancia de una excepción. Detiene la ejecución normal del método:

```
if (saldo < cantidad) {  
    throw new IllegalArgumentException("Saldo insuficiente");  
}
```

- **throws**: Se coloca en la cabecera de un método para advertir a los invocadores de que este método puede propagar (no capturar) determinadas excepciones verificadas:

```
public void leerArchivo(String ruta) throws IOException {  
    // ... código que puede lanzar IOException  
}
```

5. Diseño de Excepciones Personalizadas

En aplicaciones reales, es conveniente definir excepciones que representen errores de negocio específicos (por ejemplo, `SaldoInsuficienteException` o `UsuarioNoEncontradoException`).

- **¿Checked o Unchecked?**

- Heredamos de `Exception` para crear una excepción checked (si queremos forzar al llamador a gestionarla).
- Heredamos de `RuntimeException` para crear una excepción unchecked (recomendado para la mayoría de los casos modernos, ya que no ensucia las firmas de los métodos).

6. Ejercicios Resueltos

Ejercicio 1

Diseñar una excepción unchecked llamada `SaldoInsuficienteException` que almacene el saldo actual y el importe fallido. Implementar una clase `Cuenta` con un método `retirar(double cantidad)` que lance dicha excepción si la cantidad supera el saldo. Crear un programa que intente realizar un cobro inválido y capture la excepción personalizada mostrando los datos del saldo.

Solución:

```
// Excepción Personalizada (Unchecked)
public class SaldoInsuficienteException extends RuntimeException {
    private final double saldoActual;
    private final double importeFallido;

    public SaldoInsuficienteException(double saldoActual, double importeFallido) {
        // Llamamos al constructor de RuntimeException con un mensaje descriptivo
        super("Error: Intento de retirar " + importeFallido + " EUR con un saldo disponible de " + saldoActual + " EUR.");
        this.saldoActual = saldoActual;
        this.importeFallido = importeFallido;
    }

    public double getSaldoActual() {
        return saldoActual;
    }

    public double getImporteFallido() {
        return importeFallido;
    }
}

// Clase de Negocio
public class Cuenta {
```

```

private double saldo;

public Cuenta(double saldoInicial) {
    this.saldo = saldoInicial;
}

public void retirar(double cantidad) {
    if (cantidad <= 0) {
        throw new IllegalArgumentException("La cantidad a retirar debe ser mayor que
cero.");
    }
    if (cantidad > this.saldo) {
        // Lanzamos nuestra excepción personalizada
        throw new SaldoInsuficienteException(this.saldo, cantidad);
    }
    this.saldo -= cantidad;
    System.out.println("Retirada con éxito. Nuevo saldo: " + this.saldo + " EUR.");
}
}

// Clase Principal de Prueba
public class Main {
    public static void main(String[] args) {
        Cuenta miCuenta = new Cuenta(100.0); // Saldo inicial: 100 EUR

        try {
            miCuenta.retirar(150.0); // Intento de retirar más de lo permitido
        } catch (SaldoInsuficienteException e) {
            // Capturamos el error de negocio y accedemos a sus atributos específicos
            System.err.println("Auditoría: Transacción denegada.");
            System.err.println("Mensaje de la excepción: " + e.getMessage());
            System.err.println("Faltaban: " + (e.getImporteFallido() - e.getSaldoActual()) +
" EUR para completar la operación.");
        }
    }
}

```

(Nota: Al heredar de *RuntimeException*, el método *retirar* no necesita declarar *throws SaldoInsuficienteException* en su firma, haciendo el código más limpio, pero permitiendo igualmente la captura segura de errores mediante bloques *try-catch*).

Tema 6: Genericidad

Antes de la introducción de la genericidad en Java 5, para crear colecciones que admitiesen cualquier tipo de dato se utilizaba la clase base `Object`. Sin embargo, esto obligaba a realizar conversiones de tipo (castings) explícitas y manuales al recuperar los elementos, lo que solía provocar errores `ClassCastException` en tiempo de ejecución. La **genericidad** permite parametrizar clases, interfaces y métodos con tipos de datos definidos por el cliente en tiempo de compilación, garantizando la seguridad de tipos (*type-safety*).

1. Clases e Interfaces Genéricas

Una clase genérica se declara especificando uno o más parámetros de tipo entre caracteres de menor y mayor que (`<T>`). Por convención de Java, se utilizan letras mayúsculas como `T` (Type), `E` (Element), `K` (Key) o `V` (Value).

```
// Definición de una clase genérica
public class Caja<T> {
    private T contenido;

    public void guardar(T objeto) {
        this.contenido = objeto;
    }

    public T obtener() {
        return this.contenido;
    }
}
```

Al instanciar la clase, el cliente especifica el tipo concreto:

```
Caja<String> cajaTexto = new Caja<>(); // Caja específica para Strings
cajaTexto.guardar("Hola Mundo");
String texto = cajaTexto.obtener(); // No requiere casting explícito

Caja<Integer> cajaEntero = new Caja<>(); // Caja específica para Integers
cajaEntero.guardar(42);
```

2. Métodos Genéricos

Un método genérico define sus propios parámetros de tipo, que son independientes de los parámetros de tipo de la clase. El parámetro de tipo se declara **antes del tipo de retorno** del método:

```
public class Utilidades {  
    // Método genérico para intercambiar elementos en un array  
    public static <E> void intercambiar(E[] array, int i, int j) {  
        E temporal = array[i];  
        array[i] = array[j];  
        array[j] = temporal;  
    }  
}
```

3. Restricciones y Comodines (Wildcards)

El Comodín Indefinido (?)

Representa un tipo desconocido. Útil cuando solo leemos información de la estructura utilizando métodos heredados de `Object`.

Comodines Acotados (Bounded Wildcards)

Permiten delimitar qué tipos de datos se aceptan como parámetro genérico en base a una jerarquía de clases:

1. Cota Superior (Upper Bound): ? extends T

- Acepta la clase `T` o cualquier subclase de `T` (relación hacia abajo en la jerarquía).
- *Uso*: Principalmente para **lectura** de datos. Garantiza que cualquier elemento recuperado es al menos de tipo `T`.

2. Cota Inferior (Lower Bound): ? super T

- Acepta la clase `T` o cualquier superclase de `T` (relación hacia arriba en la jerarquía).
- *Uso*: Principalmente para **escritura** de datos. Garantiza que podemos insertar de forma segura elementos de tipo `T` en la colección.

Regla Mnemotécnica: PECS (Producer Extends, Consumer Super)

- Si la estructura de datos actúa como **productora** (de la que solo leemos elementos), usa `extends`.

- Si la estructura de datos actúa como **consumidora** (en la que solo escribimos/añadimos elementos), usa **super**.
-

4. Borrado de Tipos (Type Erasure)

La JVM no tiene soporte nativo para genéricos. Para asegurar la compatibilidad hacia atrás con el código Java antiguo, el compilador aplica el proceso de **Borrado de Tipos (Type Erasure)** durante la compilación:

1. Elimina toda la información de tipos genéricos (los parámetros `<T>`).
2. Sustituye el tipo genérico por **Object** (o por la cota especificada si existía, por ejemplo, `T extends Number` se sustituye por **Number**).
3. Inserta castings implícitos donde sea necesario para el cliente del código.

Limitaciones Críticas derivadas del Borrado de Tipos:

Debido a que en tiempo de ejecución la JVM no sabe qué es **T**, se imponen las siguientes limitaciones de diseño:

1. **No se pueden usar tipos primitivos:** No se puede hacer `Caja<int>`, se debe usar la clase envolvente (`Caja<Integer>`).
 2. **No se pueden crear instancias de T:** `T obj = new T();` da error de compilación.
 3. **No se pueden crear arrays genéricos:** `T[] array = new T[10];` da error de compilación.
 4. **No se puede hacer casting en base a genéricos en ejecución:** `if (obj instanceof Caja<String>)` da error. Solo se puede comprobar la forma cruda: `if (obj instanceof Caja)`.
-

5. Ejercicios Resueltos

Ejercicio 1

Diseñar una clase genérica `AlmacenLimitado<T extends Number>` que represente un almacén numérico que solo acepte tipos numéricos (como **Integer**, **Double**, etc.). La clase debe cumplir:

1. Almacenar una lista de elementos (usando `java.util.ArrayList`).
2. Un método `agregar(T elemento)` que añada el elemento.
3. Un método `sumarTodos()` que devuelva la suma de todos los números como un valor `double`.
4. Implementar un método estático polimórfico `imprimirAlmacen(AlmacenLimitado<?> almacen)` que muestre los datos de cualquier almacén.

Solución:

```
import java.util.ArrayList;
import java.util.List;

// Clase genérica acotada superiormente (solo acepta números)
public class AlmacenLimitado<T extends Number> {
    private List<T> elementos;

    public AlmacenLimitado() {
        this.elementos = new ArrayList<>();
    }

    public void agregar(T elemento) {
        this.elementos.add(elemento);
    }

    // Calcula la suma de todos los números en formato double
    public double sumarTodos() {
        double suma = 0.0;
        for (T elemento : elementos) {
            // Como T extiende de Number, tenemos garantizado el acceso al método
            doubleValue()
            suma += elemento.doubleValue();
        }
        return suma;
    }

    // Método estático genérico con comodín de lectura
    public static void imprimirAlmacen(AlmacenLimitado<?> almacen) {
        System.out.println("Suma del almacén: " + almacen.sumarTodos());
    }

    public static void main(String[] args) {
        // Válido: Integer extiende de Number
        AlmacenLimitado<Integer> almacenEnteros = new AlmacenLimitado<>();
        almacenEnteros.agregar(10);
        almacenEnteros.agregar(20);

        // Válido: Double extiende de Number
        AlmacenLimitado<Double> almacenReales = new AlmacenLimitado<>();
        almacenReales.agregar(5.5);
        almacenReales.agregar(4.5);

        // Error de compilación: String NO extiende de Number
        // AlmacenLimitado<String> almacenTextos = new AlmacenLimitado<>();

        System.out.println("Suma enteros:");
        imprimirAlmacen(almacenEnteros); // Imprime 30.0
    }
}
```

```
        System.out.println("Suma reales:");  
        imprimirAlmacen(almacenReales); // Imprime 10.0  
    }  
}
```

Tema 7: Especificación Formal de Código y Diseño por Contrato

En ingeniería del software, escribir código que funcione no es suficiente; debemos garantizar que el código se comporta exactamente según lo especificado. La **especificación formal** describe qué debe hacer un módulo de software de manera precisa y abstracta, separándolo de cómo se implementa internamente. Estudiaremos la metodología del **Diseño por Contrato (DbC)** y el uso de **Invariantes** para asegurar el correcto funcionamiento del software.

1. La Filosofía del Diseño por Contrato (DbC)

El Diseño por Contrato, formulado por Bertrand Meyer, establece que la interacción entre dos componentes de software (el cliente que invoca un método y el proveedor que lo implementa) se rige por un **contrato formal** que describe sus derechos y obligaciones mutuos.

Este contrato se define formalmente mediante dos elementos clave:

A. Precondición (Pre)

Es una condición lógica que **el cliente debe garantizar** antes de invocar el método.

- **Obligación del cliente:** Satisfacer la precondición.
- **Derecho del proveedor:** Asumir que la precondición es verdadera al comenzar a ejecutar el método. No está obligado a controlar qué pasa si la precondición es falsa.
- *En Java:* Si se viola una precondición, la práctica recomendada es abortar la ejecución inmediatamente lanzando una excepción de tipo unchecked (por ejemplo, `IllegalArgumentException`).

B. Postcondición (Post)

Es una condición lógica que **el proveedor garantiza** que se cumplirá una vez finalizada la ejecución del método, siempre y cuando el cliente haya cumplido la precondición inicial.

- **Obligación del proveedor:** Cumplir la postcondición.
- **Derecho del cliente:** Confiar en que el resultado final del método satisface la postcondición.

2. El Invariante de Representación (IR)

El **Invariante de Representación (IR)** de una clase es una condición lógica (un predicado booleano) sobre los atributos internos de un objeto que **debe cumplirse obligatoriamente durante todo su ciclo de vida**.

Reglas del IR:

1. Debe ser establecido por los **constructores** al crear el objeto.
2. Debe preservarse por cualquier **método público** de la clase (puede violarse temporalmente a mitad de la ejecución del método, pero al finalizar y devolver el control, el IR debe volver a ser verdadero).
3. Determina si el estado interno del objeto es **consistente** y válido.

Ejemplo conceptual de una Fracción:

Si tenemos la clase `Fraccion` con atributos `numerador` y `denominador`:

Invariante de Representación (IR): $\text{denominador} \neq 0$

3. Implementación Práctica: Programación Defensiva y `checkRep()`

Para asegurar que un objeto nunca entre en un estado inválido (violación de su IR) debido a un bug en el propio código del programador, se acostumbra a crear un método de validación interno llamado `checkRep()` (Check Representation).

Este método comprueba de forma asertiva que el IR es verdadero. Se invoca al final del constructor y al final de cualquier método mutador (setters o métodos que alteren atributos):

```
public class Fraccion {
    private int numerador;
    private int denominador;

    // Invariante de Representación (IR): denominador != 0
    private void checkRep() {
        if (this.denominador == 0) {
            throw new IllegalStateException("Violación de Invariante: El denominador no puede ser cero.");
        }
    }
}
```

```

    }
}

public Fraccion(int numerador, int denominador) {
    // Precondición: denominador != 0
    if (denominador == 0) {
        throw new IllegalArgumentException("El denominador inicial no puede ser cero.");
    }
    this.numerador = numerador;
    this.denominador = denominador;
    checkRep(); // Validar consistencia al crear
}

public void setDenominador(int denominador) {
    // Precondición: denominador != 0
    if (denominador == 0) {
        throw new IllegalArgumentException("El denominador no puede ser cero.");
    }
    this.denominador = denominador;
    checkRep(); // Validar consistencia tras mutación
}
}

```

4. Ejercicios Resueltos

Ejercicio 1

Diseñar la especificación formal e implementación de una clase `Fecha` que almacene `dia` (int) y `mes` (int). No consideraremos años bisiestos (asumiremos siempre un año común de 365 días).

1. Definir formalmente el Invariante de Representación (IR) de la clase en notación matemática.
2. Especificar formalmente (Precondición y Postcondición) un método `avanzarDia()`.
3. Implementar la clase en Java, incluyendo el método `checkRep()`.

Solución:

1. Invariante de Representación (IR):

$$\text{IR: } 1 \leq \text{mes} \leq 12 \quad \wedge \quad 1 \leq \text{dia} \leq \text{limiteDias}(\text{mes})$$

donde `limiteDias(mes)` es la función que determina la cantidad de días del mes: 31 para enero, 28 para febrero, etc.

2. Especificación formal de `avanzarDia()`:

- **Precondición:** El objeto cumple su Invariante de Representación (la fecha actual es válida).

Pre: objeto cumple IR

- **Postcondición:** La fecha se incrementa en exactamente un día. Si el día excede el límite del mes, el día pasa a 1 y el mes se incrementa. Si el mes supera 12, pasa a 1 (cambio de año). El objeto resultante sigue cumpliendo su IR.

Post: fecha resultante es la fecha del día siguiente $\wedge$ objeto cumple IR

3. Implementación en Java:

```
public class Fecha {
    private int dia;
    private int mes;

    // Retorna la cantidad de días teóricos de un mes específico (año no bisiesto)
    private static int limiteDias(int mes) {
        switch (mes) {
            case 2: return 28;
            case 4: case 6: case 9: case 11: return 30;
            default: return 31;
        }
    }

    // Invariante de Representación
    private void checkRep() {
        if (this.mes < 1 || this.mes > 12) {
            throw new IllegalStateException("Violación de Invariante: Mes fuera de rango [1, 12].");
        }
        int maxDias = limiteDias(this.mes);
        if (this.dia < 1 || this.dia > maxDias) {
            throw new IllegalStateException("Violación de Invariante: Día fuera de rango para el mes especificado.");
        }
    }

    public Fecha(int dia, int mes) {
        // Precondición: parámetros válidos
        if (mes < 1 || mes > 12 || dia < 1 || dia > limiteDias(mes)) {
            throw new IllegalArgumentException("Fecha inicial inválida.");
        }
        this.dia = dia;
        this.mes = mes;
        checkRep(); // Comprobamos que el constructor establece el IR correctamente
    }

    // Pre: objeto cumple IR (garantizado por checkRep en llamadas previas)
    // Post: la fecha avanza en 1 día y el objeto sigue cumpliendo el IR
    public void avanzarDia() {
        this.dia++;
        if (this.dia > limiteDias(this.mes)) {
            this.dia = 1;
            this.mes++;
            if (this.mes > 12) {
```

```
        this.mes = 1;
    }
}
checkRep(); // Validamos que el estado final sigue siendo consistente
}

public int getDia() { return dia; }
public int getMes() { return mes; }
}
```

Tema 8: Algoritmos Recursivos

La recursividad es una técnica de programación que permite resolver un problema definiéndolo en términos de subproblemas más sencillos de sí mismo. En lugar de utilizar bucles iterativos (`for` o `while`), una función recursiva se invoca a sí misma de forma directa o indirecta para computar el resultado final.

1. Estructura de una Función Recursiva

Para que un algoritmo recursivo sea correcto y no entre en una ejecución infinita que sature la memoria, debe estructurarse obligatoriamente en dos partes:

1. Caso Base (o Condición de Parada):

- Es la versión más simple y pequeña del problema, cuya solución se calcula de forma directa sin necesidad de realizar nuevas llamadas recursivas.
- *Importancia:* Garantiza que la recursión se detiene.

2. Caso Inductivo (o Llamada Recursiva):

- Es la llamada a la propia función pero pasándole un parámetro simplificado, reducido o modificado que esté **garantizado que se acerca al caso base**.

Ejemplo Clásico: El Factorial

Matemáticamente, el factorial de n ($n!$) se define como:

$$n! = \begin{cases} 1 & \text{si } n = 0 \quad (\text{Caso Base}) \\ n \times (n - 1)! & \text{si } n > 0 \quad (\text{Caso Inductivo}) \end{cases}$$

Implementación directa en Java:

```
public static int factorial(int n) {  
    if (n == 0) {  
        return 1; // Caso Base  
    }  
    return n * factorial(n - 1); // Caso Inductivo  
}
```

2. Tipos de Recursión

Clasificamos los algoritmos recursivos según la cantidad y la estructura de las llamadas internas:

A. Recursión Lineal

Ocurre cuando cada ejecución de la función genera, como máximo, **una única llamada recursiva** (por ejemplo, el factorial anterior o la búsqueda lineal en un array).

B. Recursión No Lineal (ej: Recursión Binaria)

Ocurre cuando una llamada genera **múltiples llamadas recursivas independientes**, ramificando la ejecución en forma de árbol.

- *Ejemplo:* El cálculo del término de Fibonacci:

$$F(n) = F(n - 1) + F(n - 2)$$

- *Problema:* Su coste temporal suele crecer de manera exponencial ($O(2^n)$) si no se optimiza, ya que repite el cálculo de los mismos subproblemas muchas veces.

C. Recursión Mutua o Cruzada

Ocurre cuando dos o más funciones se llaman entre sí de forma cíclica. Por ejemplo, la función `esPar(n)` invoca a `esImpar(n-1)` y viceversa.

D. Recursión de Cola (Tail Recursion)

Una función es recursiva de cola si la llamada recursiva es **la última instrucción exacta** que ejecuta la función, y su resultado se devuelve directamente sin realizar ninguna operación aritmética posterior.

- *En Java:* A diferencia de los lenguajes de programación funcional (como Haskell o Scala) o compiladores de C/C++, **la JVM no optimiza la recursión de cola**. Cada llamada añade un nuevo marco de activación en el Stack, por lo que una recursión muy profunda (ej. 100.000 llamadas) provocará un error de desbordamiento de pila:

`StackOverflowError`.

3. Funcionamiento de la Pila de Llamadas

(Call Stack)

La recursividad hace un uso intensivo del Stack. Cada llamada recursiva pausa la ejecución del método actual y añade un nuevo marco de activación en el tope del Stack, almacenando el estado de sus variables locales y la dirección de retorno.

Si calculamos `factorial(3)`:

```
Pila de Llamadas (Stack) en ejecución ascendente:
+-----+
3º | factorial(1): n=1, espera retorno | --> retorna 1
+-----+
2º | factorial(2): n=2, espera return 2*| --> retorna 2 * 1 = 2
+-----+
1º | factorial(3): n=3, espera return 3*| --> retorna 3 * 2 = 6
+-----+
```

Una vez que se alcanza el caso base (`factorial(1)`), la pila se empieza a desapilar de arriba a abajo, resolviendo las multiplicaciones pendientes y liberando la memoria.

4. Ejercicios Resueltos

Ejercicio 1

Implementar un algoritmo recursivo mutuo en Java para determinar si un número entero no negativo es par o impar en base a las siguientes reglas:

- Un número n es par si $n - 1$ es impar. El número 0 es par.
- Un número n es impar si $n - 1$ es par. El número 0 no es impar.

Solución:

```
public class RecursionMutua {

    // Caso Base: 0 es par.
    // Caso Inductivo: n es par si n-1 es impar.
    public static boolean esPar(int n) {
        if (n < 0) {
            throw new IllegalArgumentException("El número debe ser no negativo.");
        }
        if (n == 0) {
            return true; // Caso Base
        }
        return esImpar(n - 1); // Llamada cruzada a esImpar (Caso Inductivo)
    }
}
```

```

}

// Caso Base: 0 no es impar (retorna false).
// Caso Inductivo: n es impar si n-1 es par.
public static boolean esImpar(int n) {
    if (n < 0) {
        throw new IllegalArgumentException("El número debe ser no negativo.");
    }
    if (n == 0) {
        return false; // Caso Base
    }
    return esPar(n - 1); // Llamada cruzada a esPar (Caso Inductivo)
}

public static void main(String[] args) {
    System.out.println("¿Es 4 par?: " + esPar(4)); // Retorna true
    System.out.println("¿Es 5 par?: " + esPar(5)); // Retorna false
    System.out.println("¿Es 3 impar?: " + esImpar(3)); // Retorna true
}
}

```

(Nota: Aunque este algoritmo es muy didáctico para entender la recursión mutua, su uso práctico para comprobar paridad es ineficiente en comparación con el operador módulo $n \% 2 == 0$ de coste constante).

Tema 9: Demostración de Corrección en Bucles (Invariantes y Cotas)

La demostración formal de la corrección de un algoritmo nos permite asegurar matemáticamente que un programa se comporta exactamente según su especificación, sin importar los datos de entrada y sin depender de realizar pruebas empíricas. En este tema nos enfocaremos en bucles iterativos utilizando **Invariantes de Bucle** y **Funciones de Cota**.

1. Tripletas de Hoare y Corrección de Software

Para especificar y demostrar la corrección de un fragmento de código S , utilizamos la lógica de Hoare basada en tripletas:

$$\{Q\} S \{R\}$$

Donde:

- Q : Precondición (estado de las variables antes de ejecutar S).
- R : Postcondición (estado de las variables esperado al finalizar S).

Diferenciamos dos tipos de corrección:

- **Corrección Parcial**: Si la precondición Q es verdadera antes de ejecutar S , y **asumiendo que el programa termina**, entonces la postcondición R será verdadera.
 - **Corrección Total**: Corrección Parcial + **Garantía de Terminación** (se demuestra que el programa finalizará de forma segura en un tiempo finito).
-

2. El Invariante de Bucle (I)

Un **Invariante de Bucle** (I) es una afirmación lógica (predicado) sobre las variables de un programa que debe ser verdadera en todas las iteraciones de un bucle. Se utiliza para demostrar la **Corrección Parcial** de un bucle.

Para demostrar que un predicado I es un invariante válido, debemos verificar tres condiciones matemáticas:

1. **Inicialización (Paso Base):** El invariante debe ser verdadero inmediatamente antes de entrar al bucle, justo después de ejecutar las inicializaciones previas.

$$\{Q\} \text{ inicialización } \{I\}$$

2. **Mantenimiento (Paso Inductivo):** Si el invariante I y la condición del bucle B son verdaderos antes de una iteración, tras ejecutar el cuerpo del bucle S , el invariante I debe seguir siendo verdadero.

$$\{I \wedge B\} S \{I\}$$

3. **Finalización:** Si el bucle termina (la condición B se vuelve falsa) y el invariante I se mantiene verdadero, la conjunción de ambos debe implicar lógicamente la postcondición final R del algoritmo.

$$(I \wedge \neg B) \Rightarrow R$$

3. La Función de Cota (t)

Para demostrar la **terminación** del bucle (pasando de corrección parcial a **corrección total**), definimos una **Función de Cota (t)**, que es una expresión matemática entera en función de las variables del bucle que debe cumplir dos propiedades en cada iteración:

1. **Acotación Inferior:** Si la condición del bucle B es verdadera, la función de cota debe ser estrictamente no negativa:

$$B \Rightarrow t \geq 0$$

2. **Decrecimiento Estricto:** La ejecución del cuerpo del bucle S debe decrecer estrictamente el valor de la cota en cada iteración. Si denotamos el valor de la cota al inicio de la iteración como t_0 y al final como t :

$$\{I \wedge B \wedge t = t_0\} S \{t < t_0\}$$

Intuición: Como la cota es un número entero que decrece estrictamente en cada paso y está acotada inferiormente por 0, el bucle está obligado a terminar en un número finito de iteraciones.

4. Guía Metodológica para Diseñar Invariantes y Cotas

Ante un bucle acumulador típico que procesa un array de tamaño N desde un índice $i = 0$ hasta N :

1. **Invariante (I):** Generalmente expresa que la propiedad deseada ya se cumple de forma parcial para el subconjunto de datos procesados hasta el momento (es decir, desde el índice 0 hasta $i - 1$).
2. **Cota (t):** Mide la "distancia" o cantidad de trabajo restante para finalizar. Si el bucle termina cuando $i == N$, la cota suele ser la diferencia: $t = N - i$.

5. Ejercicios Resueltos

Ejercicio 1

Sea el siguiente fragmento de código Java diseñado para sumar los elementos de un array A de tamaño $N \geq 0$:

```
int suma = 0;
int i = 0;
while (i < N) {
    suma = suma + A[i];
    i = i + 1;
}
```

Demostrar formalmente la corrección total del bucle.

- **Precondición (Q):** $N \geq 0 \wedge A.length == N$
- **Postcondición (R):** $suma = \sum_{j=0}^{N-1} A[j]$

Solución:

1. **Proponer el Invariante (I):** El invariante indica que la variable `suma` contiene la suma acumulada de los elementos desde el índice 0 hasta $i - 1$, y que el índice i está acotado:

$$I : (suma = \sum_{j=0}^{i-1} A[j]) \quad \wedge \quad 0 \leq i \leq N$$

2. **Verificar las Condiciones del Invariante:**

- **Inicialización:** Justo antes de entrar al bucle, `suma = 0` e `i = 0`. Sustituyendo en el invariante:

$$suma = \sum_{j=0}^{-1} A[j] = 0 \quad \text{y} \quad 0 \leq 0 \leq N \quad (\text{Verdadero, pues } N \geq 0)$$

(La inicialización cumple el invariante).

- **Mantenimiento:** Asumimos que I y la condición del bucle $B : i < N$ son ciertos antes de una iteración. Tras ejecutar el cuerpo del bucle, los nuevos valores son $suma' = suma + A[i]$ e $i' = i + 1$. Comprobamos si el nuevo estado cumple el invariante:

$$suma' = suma + A[i] = \left(\sum_{j=0}^{i-1} A[j] \right) + A[i] = \sum_{j=0}^i A[j] = \sum_{j=0}^{i'-1} A[j] \quad (\text{Correcto})$$

Como $i < N \Rightarrow i + 1 \leq N \Rightarrow i' \leq N$, se cumple $0 \leq i' \leq N$. (El mantenimiento es válido).

- **Finalización:** El bucle termina cuando B es falso, es decir, $\neg(i < N) \Rightarrow i \geq N$. Por el invariante sabemos que $i \leq N$. Uniendo ambas: $i = N$. Sustituyendo $i = N$ en el invariante I :

$$suma = \sum_{j=0}^{N-1} A[j] \quad (\text{Que coincide exactamente con la Postcondición } R. \text{ Demostrado}).$$

3. Proponer y Verificar la Función de Cota (t): Proponemos la cota:

$$t = N - i$$

- **Acotación Inferior:** Debemos probar que si B es verdadero ($i < N$), entonces $t \geq 0$:

$$i < N \Rightarrow N - i > 0 \Rightarrow t \geq 0 \quad (\text{Cumple}).$$

- **Decrecimiento:** El valor de la cota tras la iteración es $t' = N - i'$. Como $i' = i + 1$:

$$t' = N - (i + 1) = N - i - 1 = t - 1 < t \quad (\text{Decrece estrictamente en cada paso}).$$

Conclusión: El bucle es formalmente correcto y tiene garantizada su terminación (Corrección Total).

Tema 10: Demostración de Corrección en Algoritmos Recursivos

La estructura de un algoritmo recursivo (Caso Base e Inductivo) se corresponde de forma directa con la estructura lógica del **Principio de Inducción Matemática**. Por tanto, la inducción es la herramienta natural y formal para demostrar que una función recursiva es correcta para cualquier valor de entrada.

1. El Principio de Inducción Matemática (Repaso)

Para demostrar que una propiedad o predicado $P(n)$ es verdadero para todo número entero $n \geq n_0$, seguimos dos pasos:

1. **Caso Base:** Demostrar que la propiedad es verdadera para el valor inicial, es decir, verificar $P(n_0)$.
2. **Paso Inductivo:**
 - **Hipótesis de Inducción (H.I.):** Asumimos que la propiedad es verdadera para cualquier valor menor que n , es decir, asumimos que $P(k)$ es cierto para todo $k < n$.
 - **Tesis Inductiva:** Demostramos que, bajo la asunción de la H.I., la propiedad también es verdadera para el elemento actual n , es decir, demostramos $P(n)$.

Si se cumplen ambos pasos, la propiedad $P(n)$ queda demostrada para todo $n \geq n_0$.

2. Metodología de Demostración para Funciones Recursivas

Dada una función recursiva en Java, el procedimiento formal consta de:

1. **Definir el Predicado de Corrección $P(n)$:** Expresar formalmente qué significa que la función sea correcta para un valor de entrada n .
 - *Ejemplo:* $P(n) : potencia(a, n) = a^n$.
2. **Verificar el Caso Base:** Analizar la rama del código que no realiza llamadas recursivas (caso de parada) y comprobar que el valor devuelto coincide con el resultado matemático esperado.

3. Demostrar el Paso Inductivo:

- Plantear la **H.I.**: Suponer que la función devuelve el resultado correcto para entradas más pequeñas (ej: $n - 1$).
- Analizar la rama recursiva del código para la entrada n . Sustituir la llamada recursiva interna por su valor teórico (justificado por la H.I.) y operar algebraicamente hasta llegar al resultado teórico esperado para n .

3. Ejemplo Práctico de Demostración Formal

Consideremos el siguiente método recursivo en Java diseñado para calcular la potencia entera no negativa a^n (con $a \neq 0$):

```
public static double potencia(double a, int n) {  
    if (n == 0) {  
        return 1.0;  
    }  
    return a * potencia(a, n - 1);  
}
```

Queremos demostrar la corrección del algoritmo para todo $n \geq 0$.

Demostración por Inducción sobre n :

1. Definición del Predicado de Corrección $P(n)$:

$P(n)$: La llamada `potencia(a, n)` termina y devuelve el valor a^n

2. Caso Base ($n = 0$):

- Evaluamos el código para $n = 0$.
- La condición `n == 0` es verdadera, por lo que el programa entra en el bloque `if` y ejecuta `return 1.0;`.
- Sabemos matemáticamente que $a^0 = 1.0$ (para $a \neq 0$).
- Por lo tanto, la función devuelve `1.0`, que coincide con a^0 . El caso base $P(0)$ es **verdadero**.

3. Paso Inductivo ($n > 0$):

- **Hipótesis de Inducción (H.I.)**: Asumimos que la función es correcta para $n - 1$, es decir, asumimos que $P(n - 1)$ es verdadero:

$$\text{potencia}(a, n - 1) = a^{n-1}$$

- **Tesis:** Debemos demostrar que entonces $P(n)$ es verdadero (la llamada `potencia(a, n)` devuelve a^n).
- Evaluamos el código para $n > 0$.
- Como $n \neq 0$, se salta el bloque `if` y se ejecuta la instrucción:

$$\text{retorno} = a \times \text{potencia}(a, n - 1)$$

- Por la Hipótesis de Inducción (H.I.), podemos sustituir la llamada recursiva `potencia(a, n-1)` por su resultado asumido a^{n-1} :

$$\text{retorno} = a \times a^{n-1}$$

- Operando algebraicamente:

$$\text{retorno} = a^1 \times a^{n-1} = a^{1+(n-1)} = a^n$$

- El valor devuelto es exactamente a^n , lo que demuestra la tesis $P(n)$.

Conclusión: Por el principio de inducción matemática, queda demostrado que el algoritmo recursivo calcula correctamente la potencia para todo $n \geq 0$.

4. Ejercicios Resueltos

Ejercicio 1

Dada la siguiente función recursiva que calcula la suma de los primeros n números enteros positivos:

```
public static int sumaN(int n) {
    if (n == 1) {
        return 1;
    }
    return n + sumaN(n - 1);
}
```

Demostrar por inducción matemática sobre n que el algoritmo es correcto para todo $n \geq 1$, es decir, que devuelve:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Solución:

1. Definir el Predicado de Corrección $P(n)$:

$$P(n) : \text{sumaN}(n) = \frac{n(n+1)}{2}$$

2. Caso Base ($n = 1$):

- Evaluamos el código para $n = 1$.
- La condición $n == 1$ se cumple, por lo que el método ejecuta `return 1;`.
- Sustituimos $n = 1$ en la fórmula teórica:

$$\frac{1(1+1)}{2} = \frac{2}{2} = 1$$

- Como el valor devuelto (1) coincide con la fórmula teórica, $P(1)$ es **verdadero**.

3. Paso Inductivo ($n > 1$):

- **Hipótesis de Inducción (H.I.):** Asumimos que $P(n-1)$ es verdadero:

$$\text{sumaN}(n-1) = \frac{(n-1)((n-1)+1)}{2} = \frac{(n-1)n}{2}$$

- **Tesis:** Demostrar que $\text{sumaN}(n) = \frac{n(n+1)}{2}$.
- Evaluamos el código para $n > 1$. Como $n \neq 1$, el método ejecuta:

$$\text{retorno} = n + \text{sumaN}(n-1)$$

- Aplicamos la H.I. sustituyendo `sumaN(n - 1)` por su valor asumido:

$$\text{retorno} = n + \frac{(n-1)n}{2}$$

- Operamos algebraicamente para unificar el denominador:

$$\text{retorno} = \frac{2n + n(n-1)}{2} = \frac{2n + n^2 - n}{2} = \frac{n^2 + n}{2} = \frac{n(n+1)}{2}$$

- El valor resultante coincide exactamente con la fórmula teórica de la tesis $P(n)$.

Conclusión: Queda demostrado por inducción formal que el método es correcto para todo $n \geq 1$.

Tema 11: Complejidad y Coste Algorítmico

Al diseñar software, no solo debemos asegurar que un algoritmo sea correcto; también debemos garantizar que sea **eficiente**. Si un programa tarda 5 segundos en procesar 100 registros pero tarda 5 días en procesar 10.000, no es viable. La teoría de la complejidad algorítmica proporciona las herramientas matemáticas para analizar de forma abstracta la cantidad de recursos (tiempo de procesamiento y espacio de memoria) que consume un algoritmo en función del tamaño de la entrada.

1. El Concepto de Operación Elemental (O.E.)

Medir el rendimiento de un programa en unidades de tiempo reales (segundos o milisegundos) es problemático porque depende de factores ajenos al algoritmo: la potencia de la CPU, el compilador, la carga del sistema operativo o el lenguaje de programación.

Para independizarnos del hardware, medimos el coste temporal calculando la cantidad de **Operaciones Elementales (O.E.)** que ejecuta el algoritmo. Una O.E. es una instrucción sencilla cuyo tiempo de ejecución está acotado superiormente por una constante (por ejemplo, asignaciones, operaciones aritméticas básicas, acceso a una posición de un array o comparaciones lógicas).

Denotamos por $T(N)$ la función que representa la cantidad de O.E. ejecutadas por el algoritmo para una entrada de tamaño N .

2. Notaciones Asintóticas

No nos interesa el valor exacto de la ecuación de coste $T(N)$ (por ejemplo, $T(N) = 3N^2 + 5N + 8$), sino su comportamiento de crecimiento a gran escala, es decir, cuando el tamaño de la entrada tiende a infinito ($N \rightarrow \infty$). Para simplificar y clasificar este comportamiento, definimos las notaciones asintóticas:

A. Cota Superior: Notación O (O Grande)

Establece una cota superior o límite máximo para el crecimiento de la función. Decimos que $f(N) \in O(g(N))$ si la función $f(N)$ crece, como máximo, tan rápido como $g(N)$ multiplicada por una constante:

$$f(N) \in O(g(N)) \Leftrightarrow \exists c > 0, N_0 > 0 \text{ tal que } f(N) \leq c \cdot g(N), \quad \forall N \geq N_0$$

B. Cota Inferior: Notación Ω (Omega)

Establece una cota inferior o límite mínimo de coste. Indica que el algoritmo tardará al menos esa cantidad:

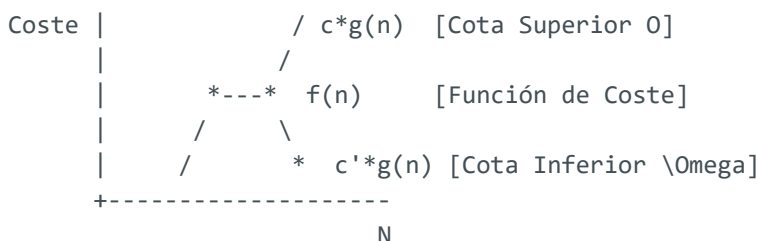
$$f(N) \in \Omega(g(N)) \Leftrightarrow \exists c > 0, N_0 > 0 \text{ tal que } f(N) \geq c \cdot g(N), \quad \forall N \geq N_0$$

C. Cota Ajustada: Notación Θ (Theta)

Establece que el coste crece exactamente con el mismo orden de magnitud que la función de referencia:

$$f(N) \in \Theta(g(N)) \Leftrightarrow f(N) \in O(g(N)) \quad \wedge \quad f(N) \in \Omega(g(N))$$

Representación de Cotas Asintóticas



Escala Común de Complejidad (de mejor a peor):

$$O(1) \subset O(\log N) \subset O(N) \subset O(N \log N) \subset O(N^2) \subset O(N^3) \subset O(2^N) \subset O(N!)$$

3. Análisis en el Peor, Mejor y Caso Medio

El número de operaciones de un algoritmo puede variar no solo por el tamaño de la entrada N , sino por la disposición concreta de los datos:

- **Peor Caso ($T_{\text{peor}}(N)$):** Es la cantidad máxima de operaciones que ejecutará el algoritmo sobre cualquier conjunto de entrada de tamaño N . Proporciona una **garantía de**

seguridad (el tiempo de ejecución real nunca superará esta cota).

- **Mejor Caso** ($T_{\text{mejor}}(N)$): Es la cantidad mínima de operaciones ejecutadas bajo la entrada más favorable.
 - **Caso Medio** ($T_{\text{medio}}(N)$): Es la esperanza matemática del coste del algoritmo, ponderando la probabilidad de ocurrencia de cada posible entrada de tamaño N . Representa el comportamiento habitual en la práctica.
-

4. Ejercicios Resueltos

Ejercicio 1

Analizar la complejidad temporal (mejor, peor y caso medio) del algoritmo de **Búsqueda Lineal** en un array no ordenado de tamaño N :

```
public static int buscar(int[] array, int valor) {
    int n = array.length;
    for (int i = 0; i < n; i++) {
        if (array[i] == valor) {
            return i; // Elemento encontrado
        }
    }
    return -1; // Elemento no encontrado
}
```

Solución:

1. Mejor Caso:

- *Escenario*: El elemento buscado se encuentra en la primera posición del array (`array[0] == valor`).
- *Análisis*: El bucle realiza exactamente una única iteración y sale de la función mediante el `return`.
- *Complejidad*: $T_{\text{mejor}}(N) = \text{cte} \Rightarrow \Theta(1)$ (Coste constante).

2. Peor Caso:

- *Escenario*: El elemento buscado no se encuentra en el array o se encuentra en la última posición (`array[N-1]`).
- *Análisis*: El bucle `for` se ejecuta completo realizando exactamente N iteraciones de coste constante.
- *Complejidad*: $T_{\text{peor}}(N) = c \cdot N + d \Rightarrow \Theta(N)$ (Coste lineal).

3. Caso Medio:

- *Asunciones*: Asumimos que el elemento está en el array y que tiene la misma probabilidad ($1/N$) de estar en cualquiera de las posiciones de la 0 a la $N - 1$.
- *Análisis*: Si el elemento está en la posición i , se realizan $i + 1$ iteraciones. La media de iteraciones es:

$$\text{Media de iteraciones} = \sum_{i=0}^{N-1} (i + 1) \cdot P(\text{estar en pos } i) = \sum_{i=0}^{N-1} (i + 1) \cdot \frac{1}{N} = \frac{1}{N} \sum_{j=1}^N j$$

Recordamos la suma de los primeros N enteros:

$$\text{Media de iteraciones} = \frac{1}{N} \cdot \frac{N(N + 1)}{2} = \frac{N + 1}{2} \approx \frac{N}{2}$$

- *Complejidad*: $T_{\text{medio}}(N) \approx c \cdot \frac{N}{2} \Rightarrow \Theta(N)$ (Coste lineal). *(Nota: El coste medio y el peor caso comparten el mismo orden de complejidad asintótica $\Theta(N)$, lo que indica que a gran escala el algoritmo crece de forma lineal en ambos escenarios).*

Tema 12: Análisis de Coste en Estructuras Iterativas

El cálculo del coste temporal en algoritmos secuenciales e iterativos (bucles) se realiza aplicando una serie de reglas algebraicas estructuradas. Veremos cómo analizar bucles simples, bucles anidados independientes y, especialmente, la resolución matemática mediante **sumatorios** para bucles anidados dependientes.

1. Reglas Operativas Fundamentales

Para evaluar el tiempo de ejecución en código iterativo, aplicamos las siguientes directrices:

A. Regla de la Secuencia

Si un bloque de código consta de dos partes consecutivas S_1 y S_2 , con costes $T_1(N)$ y $T_2(N)$ respectivamente, el coste total es su suma. Asintóticamente, domina la parte más compleja:

$$O(T_1(N) + T_2(N)) = \max(O(T_1(N)), O(T_2(N)))$$

B. Regla del Condicional (if-else)

El coste en el peor caso de una estructura condicional es el coste de evaluar la condición más el coste máximo entre sus dos ramas:

$$T_{\text{condicional}} = T(\text{condición}) + \max(T(\text{rama } \textit{try}), T(\text{rama } \textit{else}))$$

2. Análisis de Bucles Simples

El coste de un bucle es la suma de los costes de cada una de sus iteraciones. Si el coste del cuerpo es constante ($O(1)$), el análisis se reduce a contar el número de iteraciones:

Incremento Lineal (Coste Lineal)

```
for (int i = 0; i < N; i++) { /* O.E. constante */ }
```

El índice crece de 1 en 1. El bucle se ejecuta N veces. Coste: $\Theta(N)$.

Incremento Multiplicativo (Coste Logarítmico)

```
for (int i = 1; i < N; i = i * 2) { /* O.E. constante */ }
```

El índice se duplica en cada iteración ($1, 2, 4, 8, \dots, 2^k$). El bucle se detiene cuando $2^k \geq N \Rightarrow k \geq \log_2 N$. Coste: $\Theta(\log N)$.

3. Bucles Anidados Independientes

Ocurre cuando el bucle interno tiene límites de iteración constantes o que solo dependen del tamaño del problema N , sin verse afectados por el índice del bucle externo.

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
        // O.E. de coste constante c  
    }  
}
```

- El bucle externo se ejecuta N veces.
- En cada iteración del bucle externo, el bucle interno se ejecuta siempre de forma completa e idéntica N veces.
- El coste total es el producto de iteraciones:

$$T(N) = N \times N \times c = c \cdot N^2 \Rightarrow \Theta(N^2) \quad (\text{Complejidad cuadrática})$$

4. Bucles Anidados Dependientes (Uso de Sumatorios)

Ocurre cuando los límites de iteración del bucle interno dependen directamente del valor activo del índice del bucle externo. En estos casos, no podemos multiplicar directamente; debemos

plantear un **sumatorio** que represente el comportamiento variable del código y resolverlo algebraicamente.

Ejemplo Clásico:

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < i; j++) {  
        // O.E. de coste constante c  
    }  
}
```

- Cuando $i = 0$: el bucle interno se ejecuta 0 veces.
- Cuando $i = 1$: el bucle interno se ejecuta 1 vez.
- Cuando $i = 2$: el bucle interno se ejecuta 2 veces.
- ...
- Cuando $i = N - 1$: el bucle interno se ejecuta $N - 1$ veces.

Planteamiento Matemático:

$$T(N) = \sum_{i=0}^{N-1} \left(\sum_{j=0}^{i-1} c \right)$$

El sumatorio interno $\sum_{j=0}^{i-1} c$ es simplemente sumar la constante c un total de i veces, lo que resulta en $c \cdot i$:

$$T(N) = \sum_{i=0}^{N-1} (c \cdot i) = c \cdot \sum_{i=0}^{N-1} i$$

Aplicamos la fórmula matemática de la suma aritmética ($\sum_{i=0}^M i = \frac{M(M+1)}{2}$):

$$T(N) = c \cdot \frac{(N-1)N}{2} = \frac{c}{2} \cdot (N^2 - N) \Rightarrow \Theta(N^2)$$

(A pesar de que el bucle interno no se ejecuta completo todas las veces, el orden de complejidad sigue siendo cuadrático $\Theta(N^2)$).

5. Ejercicios Resueltos

Ejercicio 1

Calcular la complejidad temporal en el peor caso para el siguiente fragmento de código:

```
int k = 0;
for (int i = 0; i < N; i++) {
    for (int j = N; j > i; j = j / 2) {
        k++;
    }
}
```

Solución:

1. **Analizar el bucle interno:** El índice j empieza en N y decrece dividiéndose por 2 ($j = j / 2$) en cada paso hasta ser menor o igual a i . El número de iteraciones en función de i viene dado por la cantidad de veces que podemos dividir el rango entre N e i por 2. Esto equivale a:

$$\text{Iteraciones internas} \approx \log_2 \left(\frac{N}{i+1} \right) = \log_2 N - \log_2(i+1)$$

2. **Plantear el sumatorio del coste total:**

$$T(N) = \sum_{i=0}^{N-1} \left(\log_2 \left(\frac{N}{i+1} \right) \right) = \sum_{i=0}^{N-1} (\log_2 N - \log_2(i+1))$$

Separando los sumatorios:

$$T(N) = \left(\sum_{i=0}^{N-1} \log_2 N \right) - \sum_{i=0}^{N-1} \log_2(i+1)$$

- La primera parte es sumar la constante $\log_2 N$ un total de N veces: $N \log_2 N$.
- La segunda parte es $\sum_{i=0}^{N-1} \log_2(i+1) = \log_2(1) + \log_2(2) + \dots + \log_2(N) = \log_2(N!)$. Utilizando la **aproximación de Stirling** para el logaritmo del factorial: $\log_2(N!) \approx N \log_2 N - N \log_2 e$. Sustituyendo:

$$T(N) \approx N \log_2 N - (N \log_2 N - N \log_2 e) = N \log_2 e \implies \Theta(N)$$

Conclusión: Aunque contenga un bucle interno dependiente con divisiones, el orden de complejidad asintótico de este algoritmo es lineal $\Theta(N)$.

Tema 13: Análisis de Coste en Algoritmos Recursivos

El análisis de la complejidad temporal de las funciones recursivas es más complejo que el de las iterativas, ya que no disponemos de bucles explícitos que podamos contar o transformar en sumatorios directos. En su lugar, el coste de un algoritmo recursivo $T(N)$ se define mediante una **ecuación de recurrencia** (una ecuación que se define en términos de sí misma).

Estudiaremos cómo plantear estas ecuaciones, cómo resolverlas por expansión y cómo aplicar el potente **Teorema Máster**.

1. Formulación de Ecuaciones de Recurrencia

Una ecuación de recurrencia para un coste $T(N)$ consta siempre de dos partes:

1. **Caso Base:** El coste de ejecutar el caso de parada (que suele ser un coste constante c_0).
2. **Caso Inductivo:** El coste del cuerpo del método, que incluye el coste del código no recursivo ($f(N)$, para dividir y combinar el problema) más el coste de las llamadas recursivas sobre subproblemas más pequeños.

Ejemplo: Búsqueda Binaria

El algoritmo divide el array por la mitad en cada paso, realiza una comparación constante y hace una única llamada recursiva sobre la mitad seleccionada:

- Caso Base ($N = 1$): $T(1) = c_0$ (coste constante de comparar).
 - Caso Inductivo ($N > 1$): $T(N) = T\left(\frac{N}{2}\right) + c_1$ (llamada sobre la mitad del tamaño más coste de comparación constante).
-

2. Resolución por el Método de Expansión (o Sustitución)

Consiste en sustituir sucesivamente la definición de la función en su propio caso recursivo para detectar un patrón matemático en función del paso de expansión k :

Ejemplo: Factorial Recursivo

Ecuación de recurrencia:

- $T(0) = c_0$
- $T(N) = T(N - 1) + c_1$ (para $N > 0$)

Expansión paso a paso:

- Paso 1: $T(N) = T(N - 1) + c_1$
- Paso 2: Como $T(N - 1) = T(N - 2) + c_1$, sustituimos:

$$T(N) = [T(N - 2) + c_1] + c_1 = T(N - 2) + 2c_1$$

- Paso 3: $T(N) = T(N - 3) + 3c_1$
- Paso k : Detectamos el patrón general:

$$T(N) = T(N - k) + k \cdot c_1$$

El proceso se detiene cuando alcanzamos el caso base, es decir, cuando la entrada del término recursivo es 0:

$$N - k = 0 \implies k = N$$

Sustituyendo $k = N$ en la ecuación del paso k :

$$T(N) = T(0) + N \cdot c_1 = c_0 + c_1 \cdot N \implies \Theta(N) \quad (\text{Coste lineal})$$

3. El Teorema Máster

El Teorema Máster proporciona una "receta" matemática directa para resolver ecuaciones de recurrencia basadas en la estrategia de **Divide y Vencerás**, donde el problema se divide en a subproblemas de tamaño N/b , y el coste de dividir y combinar los resultados es de orden polinomial $\Theta(N^k)$.

La ecuación general debe tener la forma:

$$T(N) = a \cdot T\left(\frac{N}{b}\right) + \Theta(N^k)$$

Donde $a \geq 1$, $b > 1$, y $k \geq 0$. Comparamos el valor de a con el término b^k :

Caso 1: $a > b^k$ (Domina la recursión)

El coste principal reside en la gran cantidad de llamadas recursivas secundarias.

$$T(N) \in \Theta(N^{\log_b a})$$

Caso 2: $a = b^k$ (Coste equilibrado)

El coste de las llamadas recursivas y el coste de combinación están en equilibrio.

$$T(N) \in \Theta(N^k \cdot \log N)$$

Caso 3: $a < b^k$ (Domina la combinación)

El coste principal reside en el trabajo de dividir y combinar los subproblemas (N^k).

$$T(N) \in \Theta(N^k)$$

4. Ejemplos de Aplicación del Teorema Máster

A. Búsqueda Binaria

- Ecuación: $T(N) = 1 \cdot T(N/2) + \Theta(1)$
- Parámetros: $a = 1, b = 2, k = 0$ (pues $\Theta(1) = \Theta(N^0)$).
- Comparación: $b^k = 2^0 = 1$.
- Caso del Teorema: Como $a = b^k$ ($1 = 1$), aplica el **Caso 2**:

$$T(N) \in \Theta(N^0 \cdot \log N) \Rightarrow \Theta(\log N)$$

B. Ordenación por Mezcla (MergeSort)

- Ecuación: $T(N) = 2 \cdot T(N/2) + \Theta(N)$ (dos llamadas a la mitad del tamaño más coste lineal de mezclar).
- Parámetros: $a = 2, b = 2, k = 1$ (pues $\Theta(N) = \Theta(N^1)$).
- Comparación: $b^k = 2^1 = 2$.
- Caso del Teorema: Como $a = b^k$ ($2 = 2$), aplica el **Caso 2**:

$$T(N) \in \Theta(N^1 \cdot \log N) \Rightarrow \Theta(N \log N)$$

5. Ejercicios Resueltos

Ejercicio 1

Calcular la complejidad asintótica de un algoritmo cuya ecuación de recurrencia de coste es:

- $T(1) = c$
- $T(N) = 3 \cdot T\left(\frac{N}{2}\right) + n^2$

Solución:

1. **Identificar parámetros para el Teorema Máster:** La ecuación tiene la forma $T(N) = a \cdot T(N/b) + \Theta(N^k)$.

- $a = 3$ (se generan 3 subproblemas).
- $b = 2$ (el tamaño de cada subproblema se divide por 2).
- $k = 2$ (el trabajo de combinación es cuadrático, N^2).

2. **Comparar a con b^k :**

- $a = 3$
- $b^k = 2^2 = 4$ Comparando ambos valores:

$$a < b^k \quad (3 < 4)$$

3. **Aplicar el caso del Teorema:** Al cumplirse que $a < b^k$, estamos en el **Caso 3** (el coste está dominado por el trabajo de combinación no recursivo de la función principal):

$$T(N) \in \Theta(N^k) \Rightarrow \Theta(N^2)$$

Conclusión: La complejidad temporal asintótica del algoritmo en el peor caso es cuadrática $\Theta(N^2)$.

Tema 14: Tipos Abstractos de Datos (TADs) Lineales

Un Tipo Abstracto de Datos (TAD) es una especificación matemática de un conjunto de datos junto con las operaciones permitidas sobre ellos, definida de manera totalmente independiente de cómo se codifiquen o almacenen físicamente en memoria. Estudiaremos las estructuras lineales clásicas (**Pilas**, **Colas** y **Listas**) analizando sus interfaces y sus implementaciones eficientes en Java mediante arrays y nodos dinámicos.

1. Abstracción en Estructuras de Datos

En Java, la separación entre la definición de un TAD y su implementación física se modela de forma natural:

- **La Interfaz (`interface`)**: Define la especificación abstracta de los métodos (qué hace el TAD).
- **La Clase (`class`)**: Define la estructura de datos física y codifica los métodos (cómo lo hace).

Las tres estructuras lineales clásicas son:

1. **Pila (`Stack`)**: Estructura LIFO (Last-In, First-Out). El último elemento en entrar es el primero en salir. (Ej: historial de navegación, deshacer en editores, pila de llamadas de la JVM).
 2. **Cola (`Queue`)**: Estructura FIFO (First-In, First-Out). El primer elemento en entrar es el primero en salir. (Ej: cola de impresión, buffer de paquetes de red).
 3. **Lista (`List`)**: Colección secuencial ordenada donde se pueden insertar y eliminar elementos en cualquier posición de la secuencia.
-

2. Alternativas de Implementación: Estática vs. Dinámica

Para cualquier TAD lineal, existen dos estrategias básicas de representación en memoria:

A. Implementación Estática (Basada en Arrays)

- **Estructura:** Los elementos se guardan en posiciones contiguas de un array de tamaño fijo.
- **Ventajas:** Acceso aleatorio directo muy rápido por índice ($\Theta(1)$). Menor consumo de memoria al no requerir punteros de enlace.
- **Desventajas:** Capacidad limitada fija al crear la estructura. Si el array se llena, se debe crear uno nuevo de mayor tamaño y copiar todos los elementos ($\Theta(N)$).

B. Implementación Dinámica (Basada en Nodos Enlazados)

- **Estructura:** Los elementos se guardan en objetos independientes (nodos) dispersos en el Heap. Cada nodo almacena el dato útil y una **referencia (puntero)** al siguiente nodo.
- **Ventajas:** Capacidad ilimitada y dinámica. La inserción o eliminación al inicio de la estructura es extremadamente eficiente ($\Theta(1)$) y no requiere desplazar elementos en memoria.
- **Desventajas:** No permite acceso directo; para acceder al elemento i -ésimo es necesario recorrer la cadena de nodos desde el primero ($\Theta(N)$). Consume más memoria debido a las referencias de enlace.

3. Implementación Completa de una Pila Dinámica Genérica

Definiremos el contrato del TAD Pila y su correspondiente implementación dinámica utilizando nodos enlazados en Java.

A. La Interfaz del TAD Pila (**PilaTAD**)

```
public interface PilaTAD<T> {  
    // Inserta un elemento en el tope de la pila  
    void apilar(T elemento);  
  
    // Elimina y devuelve el elemento del tope de la pila.  
    // Precondición: la pila no debe estar vacía.  
    T desapilar();  
  
    // Devuelve el elemento del tope sin eliminarlo.  
    // Precondición: la pila no debe estar vacía.  
    T cima();  
  
    // Retorna true si la pila no contiene elementos
```

```
    boolean estaVacia();  
}
```

B. La Implementación Dinámica (**PilaEnlazada**)

```
import java.util.EmptyStackException;  
  
public class PilaEnlazada<T> implements PilaTAD<T> {  
  
    // Clase interna privada que representa el nodo de enlace  
    private static class Nodo<E> {  
        E dato;  
        Nodo<E> siguiente;  
  
        Nodo(E dato, Nodo<E> siguiente) {  
            this.dato = dato;  
            this.siguiente = siguiente;  
        }  
    }  
  
    // Atributo: referencia al nodo que está en el tope de la pila  
    private Nodo<T> tope;  
  
    public PilaEnlazada() {  
        this.tope = null; // Pila inicialmente vacía  
    }  
  
    @Override  
    public void apilar(T elemento) {  
        // Creamos un nuevo nodo cuyo siguiente es el antiguo tope,  
        // y actualizamos el tope de la pila. Coste: Theta(1).  
        this.tope = new Nodo<>(elemento, this.tope);  
    }  
  
    @Override  
    public T desapilar() {  
        // Precondición del contrato  
        if (estaVacia()) {  
            throw new EmptyStackException();  
        }  
        T datoRecuperado = this.tope.dato;  
        this.tope = this.tope.siguiente; // Movemos el puntero al siguiente nodo. Coste:  
        Theta(1).  
        return datoRecuperado;  
    }  
  
    @Override  
    public T cima() {  
        // Precondición del contrato  
        if (estaVacia()) {  
            throw new EmptyStackException();  
        }  
        return this.tope.dato; // Coste: Theta(1).  
    }  
  
    @Override  
    public boolean estaVacia() {
```

```
        return this.tope == null;
    }
}
```

4. Ejercicios Resueltos

Ejercicio 1

Utilizar el TAD Pila para implementar un algoritmo que compruebe si una cadena de texto que contiene paréntesis, llaves y corchetes está balanceada correctamente. Por ejemplo:

- "[{()}]" -> Válida (retorna **true**).
- "[{()]}" -> Inválida (retorna **false**).

Solución:

```
public class BalanceadorSimbólico {

    public static boolean estaBalanceado(String expresion) {
        PilaTAD<Character> pila = new PilaEnlazada<>();

        for (int i = 0; i < expresion.length(); i++) {
            char c = expresion.charAt(i);

            // Si es un símbolo de apertura, lo apilamos
            if (c == '(' || c == '{' || c == '[') {
                pila.apilar(c);
            }
            // Si es un símbolo de cierre, comprobamos el tope
            else if (c == ')' || c == '}' || c == ']') {
                if (pila.estaVacia()) {
                    return false; // Cierre sin apertura previa
                }
                char tope = pila.desapilar();
                if (!sonPareja(tope, c)) {
                    return false; // Los símbolos no coinciden
                }
            }
        }
        // La cadena está balanceada si no quedan símbolos pendientes en la pila
        return pila.estaVacia();
    }

    private static boolean sonPareja(char apertura, char cierre) {
        return (apertura == '(' && cierre == ')') ||
            (apertura == '{' && cierre == '}') ||
            (apertura == '[' && cierre == ']');
    }

    public static void main(String[] args) {
        System.out.println(estaBalanceado("[{()}]")); // Imprime true
    }
}
```

```
        System.out.println(estaBalanceado("{[()]}" )); // Imprime false
        System.out.println(estaBalanceado("((()))" )); // Imprime false
    }
}
```

(Nota: Este algoritmo tiene un coste temporal lineal $\Theta(N)$ y espacial $O(N)$ en el peor caso, lo cual es óptimo para la resolución del problema de balanceo de paréntesis).

Glosario de Términos

- **Borrado de Tipos (Type Erasure):** Proceso mediante el cual el compilador de Java elimina toda la información de tipos genéricos (`()`) y la sustituye por `Object` o por su cota superior durante la generación de bytecode para mantener compatibilidad hacia atrás.
- **Clase Abstracta:** Clase declarada con `abstract` que representa un concepto incompleto y no se puede instanciar directamente. Puede contener métodos abstractos (sin implementar).
- **Comodín (Wildcard):** Símbolo de interrogación (`?`) en genericidad que representa un tipo desconocido. Permite definir covarianza (`? extends T`) y contravarianza (`? super T`) siguiendo la regla PECS.
- **Diseño por Contrato (DbC):** Metodología que formaliza la interacción de módulos usando condiciones lógicas: Precondiciones (obligación del cliente) y Postcondiciones (obligación del proveedor).
- **Enlace Dinámico (Dynamic Binding):** Mecanismo de resolución en tiempo de ejecución por el cual la JVM selecciona la implementación concreta de un método en base al tipo dinámico del objeto sobre el que se invoca.
- **Excepción Verificada (Checked Exception):** Excepción comprobada por el compilador en tiempo de compilación. Obliga al desarrollador a gestionarla en un bloque try-catch o a declararla en la cabecera mediante `throws`.
- **Garbage Collector (GC):** Hilo de baja prioridad de la JVM que libera de forma automática la memoria ocupada por objetos en el Heap que han quedado inalcanzables en la pila de referencias.
- **Heap (Montículo):** Área de memoria dinámica global de la JVM donde se crean y almacenan todos los objetos y arrays durante la ejecución.
- **Invariante de Bucle (I):** Predicado lógico sobre las variables de un programa que debe cumplirse obligatoriamente en la inicialización, en cada iteración y al finalizar un bucle iterativo.
- **Invariante de Representación (IR):** Condición lógica sobre los atributos internos de un objeto que garantiza su consistencia y validez. Debe cumplirse al final de constructores y de cualquier método público.
- **Notación Asintótica (O , Ω , Θ):** Notaciones matemáticas usadas para clasificar el ritmo de crecimiento del coste de ejecución (tiempo o espacio) de un algoritmo cuando la entrada tiende a infinito ($N \rightarrow \infty$).
- **Polimorfismo:** Capacidad de una referencia de clase base para apuntar a objetos de cualquiera de sus clases derivadas de forma transparente.
- **Recursión de Cola (Tail Recursion):** Tipo de recursión donde la llamada recursiva es la última instrucción exacta que ejecuta la función, sin realizar operaciones pendientes tras el retorno.

- **Stack (Pila):** Región de memoria secuencial y LIFO de la JVM donde se almacenan las variables locales de tipos primitivos, los marcos de activación de métodos y las referencias a objetos del Heap.
- **Teorema Máster:** Herramienta matemática que permite resolver ecuaciones de recurrencia de tipo "divide y vencerás" ($T(N) = aT(N/b) + \Theta(N^k)$) en coste temporal de forma directa.
- **Tipo Abstracto de Datos (TAD):** Especificación abstracta que define un conjunto de datos y sus operaciones de forma matemática, independiente de su implementación física en memoria.

Bibliografía Recomendada

1. **Meyer, B. (1997).** *Object-Oriented Software Construction* (2nd ed.). Prentice Hall.
 - *Nota:* La obra seminal sobre orientación a objetos y Diseño por Contrato escrita por su propio creador.
2. **Bloch, J. (2018).** *Effective Java* (3rd ed.). Addison-Wesley.
 - *Nota:* La guía de referencia imprescindible de mejores prácticas de diseño de clases, excepciones, genéricos y colecciones en Java.
3. **Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983).** *Data Structures and Algorithms*. Addison-Wesley.
 - *Nota:* El texto clásico académico más influyente sobre el análisis de coste, complejidad y diseño de Tipos Abstractos de Datos.
4. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009).** *Introduction to Algorithms* (3rd ed.). MIT Press.
 - *Nota:* Considerada la "biblia" mundial en algoritmos, ideal para profundizar en ecuaciones de recurrencia y el Teorema Máster de forma rigurosa.